

Investing Accelerator Summit - Product Design and Frontend Architecture

Next.js information architecture, subscription journeys, interface states, API contracts, and production debugging

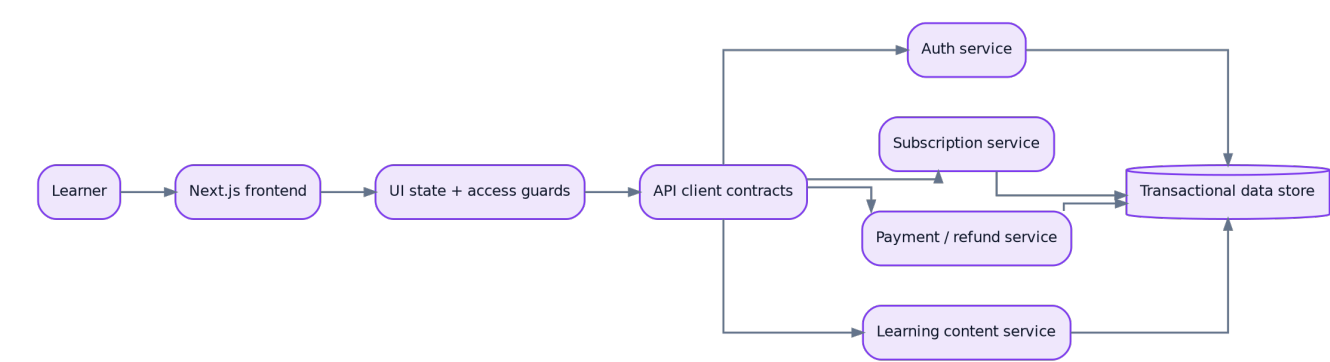
Project context: Software Engineer Intern, Investing Accelerator Summit, January-February 2026	Status: Internship contribution case study
---	---

A product-design and frontend-architecture case study for a subscription-based stock-market learning platform. The contribution centered on organizing the Next.js frontend, defining pages and reusable components, mapping payment, refund, cancellation, and subscription states, specifying the API interactions required by those journeys, and supporting production debugging through application logs and data-flow analysis. This document does not claim that the intern independently implemented the payment APIs.

Technology stack

Next.js	React	Component architecture	Responsive UI
API contract design	Subscription workflows	Transactional states	Logs and debugging

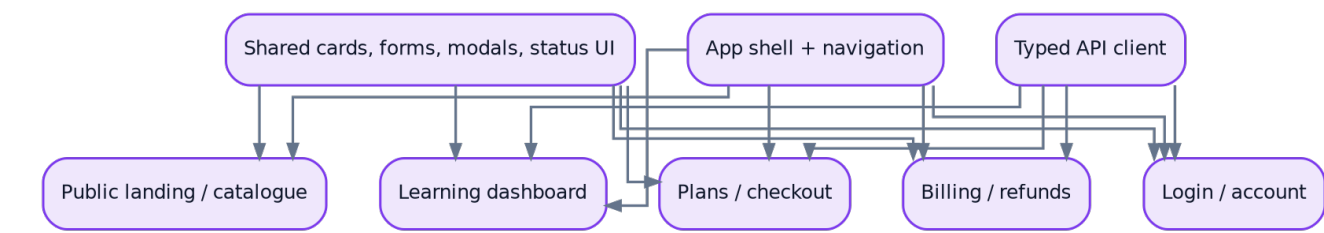
System at a glance



Learning experience design Organizes discovery, course/content access, dashboards, and account status into a coherent user journey.	Subscription interface Designs plan comparison, checkout handoff, pending, success, failure, active, cancelled, and expired states.
Refund and cancellation UX Makes eligibility, confirmation, processing, completion, and failure visible to the user.	Operational feedback Defines loading, empty, validation, retry, and support states informed by logs and production failures.

System design and component ownership

The contribution is best represented as a frontend system boundary. Next.js pages and reusable UI components own presentation and navigation. A typed API-client layer translates interface actions into backend requests. Authentication, subscriptions, payments, refunds, content access, and persistence are backend responsibilities that the frontend consumes through defined contracts.



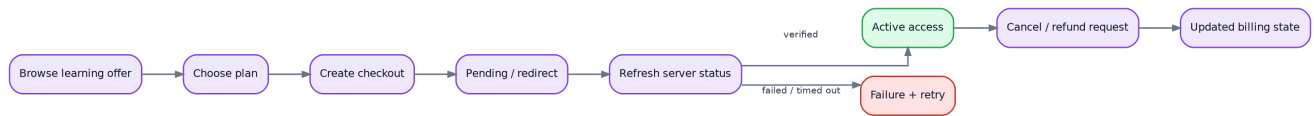
Component	Responsibility
Route and page structure	Landing, authentication, learning catalogue, content detail, dashboard, plans, billing, and account pages.
Shared design system	Navigation, cards, forms, modals, status banners, tables, skeletons, and responsive layout primitives.
Frontend state layer	Holds the current user, entitlement status, selected plan, request state, and recoverable errors.
API client contracts	Defines the requests and responses needed for plans, checkout, subscription status, cancellation, and refund.
Access guards	Controls navigation and content based on authentication and current entitlement state.
Diagnostics	Uses request context and application logs to trace failed workflows and improve user-visible recovery states.

Key design decisions

- Model the complete subscription lifecycle instead of treating payment as a single success button.
- Keep payment-provider details behind backend API contracts; the browser should receive normalized status.
- Use reusable status, confirmation, error, and loading components across payment and account journeys.
- Define API requirements from the user flow before backend implementation so missing states are discovered early.

Core execution flow

A learner moves from plan selection to a backend-created checkout session. The browser then waits for a normalized subscription state rather than trusting a client callback alone. Cancellation and refund are separate workflows with confirmation, eligibility, processing, and final-result states.



Implementation notes

- Built and organized frontend architecture using Next.js concepts: pages, components, navigation, API calls, and state updates.
- Mapped payment, refund, cancellation, and subscription workflows into explicit interface states.
- Supported application data flow and persistence discussions so transactional records remained consistent across the experience.
- Monitored logs and debugged production issues with cross-functional team members.
- The user previously clarified that the APIs themselves were not implemented by them; this case study therefore focuses on design, frontend, and integration contracts.

Representative Next.js frontend contract

```
type SubscriptionState =  
  | "none" | "checkout_pending" | "active"  
  | "cancel_pending" | "cancelled"  
  | "refund_pending" | "refunded" | "failed";  
  
async function choosePlan(planId: string) {  
  setView({ state: "checkout_pending" });  
  const checkout = await api.createCheckout({ planId });  
  router.push(checkout.redirectUrl);  
}  
  
async function refreshEntitlement() {  
  const status = await api.getSubscriptionStatus();  
  setView(mapBackendStatusToUi(status));  
}
```

Representative architecture pattern, not private company source code. It illustrates the frontend state and API boundary described in the internship contribution.

Engineering review and production path

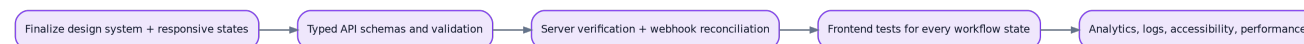
Engineering strengths

- Focuses on full user journeys, including failure and recovery states.
- Separates frontend responsibility from payment verification and transactional persistence.
- Connects product design decisions to reusable frontend architecture.
- Includes practical production-debugging and log-analysis experience.

Current limitations

- Private company source code, designs, and production data are not reproduced.
- The intern did not independently implement the payment and refund APIs.
- A production payment flow still requires server-side verification, idempotency, webhook processing, and reconciliation.
- Exact framework code and visual screenshots are unavailable in the public material used for this sample.

Recommended next steps



Why this is relevant to AI training work

- Demonstrates decomposition of ambiguous product requirements into explicit states and contracts.
- Creates high-value test cases for payment success, timeout, duplicate callback, cancellation, refund, and stale entitlement states.
- Shows disciplined provenance by separating implemented contribution from backend work owned by others.
- Provides a clear specification against which generated frontend code can be evaluated.

Profile reference:

<https://www.linkedin.com/in/sidheshwar-sarangal-0b31482b8/>

Scope and provenance:

Prepared from the internship entry in the supplied resume and the user's earlier clarification that their core contribution was product/frontend architecture, workflow design, API requirement definition, logs, and production debugging. No private company code or confidential data is included.

